第28课 二分查找

一、课程信息

项目	内容
课次	第 28 课 / 共 40 课
主题	二分查找
前置知识	数组、排序（冒泡/选择/sort）、循环（for / while）
后续衔接	第 29 课 —— 递归思想（一）

教学目标（可检验）

1. **能说出**二分查找的前提条件（数组必须有序），并能口头解释为什么无序数组不能用二分查找。
2. **能独立默写**一段完整的二分查找代码（给定有序数组，查找目标值，返回下标或 -1），编译通过且结果正确。
3. **能在限时内完成**至少 2 道课堂练习题，正确使用 `left`、`right`、`mid` 三个变量完成查找。

二、教学重点与难点

重点

- 理解"每次砍掉一半"的核心思想。
- 掌握 `left`、`right`、`mid` 三个变量的含义与更新规则。
- 掌握 `while(left <= right)` 循环条件的含义。

难点

- **边界更新**：什么时候 `left = mid + 1`，什么时候 `right = mid - 1`，容易混淆或漏写 `+1` / `-1`。
- **循环条件**：为什么是 `<=` 而不是 `<`，学生容易写错导致漏查或死循环。
- **有序前提**：学生容易忘记先排序就直接二分，导致结果错误。

三、课堂时间分配（共 100 分钟）

时间	时长	环节	内容
0′–5′	5 min	热身游戏	"猜数字"游戏导入：老师心里想一个 1-100 的数，请一位同学来猜
5′–20′	15 min	新知讲解（一）	顺序查找 vs 二分查找：效率对比 + 生活比喻
20′–30′	10 min	例题 1	带学生手写第一段二分查找代码，逐行讲解
30′–45′	15 min	课堂练习 1	学生独立完成练习题 1，老师巡视辅导
45′–55′	10 min	互动游戏	全班一起玩"二分猜数"：学生当裁判，老师用二分法猜再反过来
55′–70′	15 min	新知讲解（二）+ 例题 2	查找不存在的数 / 找第一个 ≥ x 的位置（进阶）
70′–90′	20 min	课堂练习 2-3	学生完成剩余练习题，老师答疑
90′–100′	10 min	总结 + 作业布置 + 下节课预告	回顾要点、易错提醒、布置洛谷作业

> **讲解总时长**：15 + 10 + 15 = 40 分钟（单次最长 15 分钟，不超过 25 分钟上限）
> **练习总时长**：15 + 20 = 35 分钟

四、讲稿提纲

4.1 热身导入——猜数字游戏（5 分钟）

> 同学们，我们来玩一个游戏！老师心里想了一个 1 到 100 之间的数字，你们来猜。每次猜完，老师只会告诉你"大了"或者"小了"。谁能用最少的次数猜到？

- 请一位同学上台来猜。（通常学生会随机猜，需要很多次）
- 再请一位同学，提示他："你能不能每次都猜**中间**的那个数？"
- 引导发现：每次猜中间的数，不管大了还是小了，都能**砍掉一半**的可能！
- **结论**：1 到 100 的数字，最多只要猜 **7 次**就能猜到！（因为 $2^7 = 128 > 100$）

> 这种"每次砍掉一半"的方法，有一个很厉害的名字，叫做——**二分查找**！

4.2 顺序查找 vs 二分查找（15 分钟）

4.2.1 顺序查找——翻字典的"笨办法"

> 想象一下，你要在一本 1000 页的字典里找一个字。如果你从第 1 页开始，一页一页往后翻，最倒霉的情况要翻多少页？——**1000 页**！这就叫**顺序查找**，一个一个找。

用代码回顾：

```
// 顺序查找：一个一个找
int find(int a[], int n, int x) {
 for (int i = 0; i < n; i++) {
 if (a[i] == x) return i; // 找到了，返回下标
 }
 return -1; // 没找到
}
```

> 顺序查找就像排队买奶茶，前面有 100 个人，你就得等 100 轮。**慢！**

4.2.2 二分查找——翻字典的"聪明办法"

> 但是！字典里的字是**按拼音顺序排好的**。所以你可以先翻到中间，看看要找的字在前半本还是后半本，然后再翻剩下那半本的中间……每次都砍掉一半！1000 页的字典，最多翻 **10 次**就能找到！（因为 $2^{10} = 1024 > 1000$）

效率对比表（板书 / PPT）：

数据量	顺序查找（最多次数）	二分查找（最多次数）
10	10	4
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20

> 一百万个数，顺序查找要找一百万次，二分查找只要 **20 次**！是不是超级厉害？

4.2.3 二分查找的"规矩"——必须有序

> 但是二分查找有一个**非常重要的前提**：数组必须是**排好序的**！
>
> 为什么呢？想想看，如果字典里的字是乱排的，你翻到中间一页，看到的字比你要找的字大，你能确定要找的字在前半本吗？——不能！因为后半本也可能有更小的字。所以如果是乱的，二分查找就**失灵**了。
>
> 记住这句口诀：**"二分查找，有序才行！"**

4.3 二分查找代码讲解（10 分钟）

> 现在我们来看看二分查找的代码长什么样。别怕，只有几行！

关键概念（板书画图）：

```
数组：[2, 5, 8, 12, 16, 23, 38, 42, 56, 72]

 left mid right

要找的数 x = 23
```

> 想象你面前有一排排好序的数字卡片：
> - `left` （左手）指着最左边
> - `right` （右手）指着最右边
> - `mid` （眼睛看的位置）在正中间
>
> 你看了一眼 mid 位置的数：
> - 如果**刚好等于** x 太棒了，找到了！
> - 如果 mid 位置的数**比 x 小** 说明 x 在**右半边**，左手往右移（`left = mid + 1`）
> - 如果 mid 位置的数**比 x 大** 说明 x 在**左半边**，右手往左移（`right = mid - 1`）
>
> 每次移完手，范围就少了一半！直到找到，或者左手跑到右手的右边去了（说明没找到）。

4.4 互动游戏——二分猜数（10 分钟）

玩法一：老师想数（写在纸上不给学生看），全班一起用二分法猜。

- 范围 1-100，老师只回答"大了""小了""对了"。
- 要求学生每次都猜 `(left + right) / 2`。
- 板书记录每次的 left、right、mid，让学生直观看到范围在缩小。

玩法二：请一位同学想数，老师来猜（故意展示二分过程），其余同学验证老师有没有算错 mid。

> 你们看，老师最多 7 次就猜到了！以后你们和朋友玩猜数字游戏，用这招，赢定了！

七、常见错误与应对

错误 1：循环条件写成 `left < right`（漏了等号）

```
// 错误写法
while (left < right) { ... }

// 正确写法
while (left <= right) { ... }
```

后果：当 `left == right` 时，还有一个数没有检查就退出了，导致漏查。

课堂应对：让学生在纸上模拟一个只有 1 个元素的数组查找过程，发现用 `<` 会直接跳过。

> 记忆口诀：**"左小于等于右，一个都不能漏！"**

错误 2：边界更新忘记 +1 / -1

```
// 错误写法
left = mid; // 没有 +1
right = mid; // 没有 -1

// 正确写法
left = mid + 1;
right = mid - 1;
```

后果：`mid` 已经比较过了还留在范围里，可能导致**死循环**（left 和 right 不再变化）。

课堂应对：故意演示一次死循环，让学生看到程序"卡住了"，印象深刻。

> 比喻：你翻字典看了第 50 页，发现不是，下次还从第 50 页开始翻？那你永远也翻不完！

错误 3：对无序数组使用二分查找

```
// 数组没有排序就二分
int a[] = {5, 2, 8, 1, 9, 3};
// 直接二分查找 结果不可靠！
```

课堂应对：先给学生一个无序数组，让他们尝试二分查找一个存在的数，发现找不到。

> **口诀："二分之前先排序，无序二分白忙活！"**

错误 4：mid 计算溢出（了解即可）

```
// 当 left 和 right 都很大时，left + right 可能超出 int 范围
// 更安全的写法：
int mid = left + (right - left) / 2;
```

课堂处理：简单提一句即可，告诉学生 CSP-J 的数据范围一般不会触发这个问题，但知道有这回事。

错误 5：找到后忘记 break（基础版）

```
if (a[mid] == x) {
 ans = mid;
 // 忘记写 break；程序还会继续循环，虽然结果可能对，但浪费时间
}
```

课堂应对：提醒学生基础版找到就可以停了，养成好习惯。

九、下节课预告

> 同学们，今天我们学了二分查找——"每次砍一半"的查找神器。下节课我们要学一个更有趣的东西：**递归**！
>
> 什么是递归呢？给你们讲个小故事：
>
> 从前有座山，山里有座庙，庙里有个老和尚在给小和尚讲故事。讲的什么故事呢？——"从前有座山，山里有座庙，庙里有个老和尚在给小和尚讲故事……"
>
> 这种"自己调用自己"的方法就叫递归！下节课我们会学到：
> - 什么是递归
> - 递归的"出口"（不然就像那个故事一样永远讲不完了！）
> - 用递归解决一些有趣的问题
>
> 回家好好完成作业，我们下节课见！

教案结束